

Documentación técnica integral — Jahatelo

Revisión: 2026-08-05

Alcance: web pública, PWA, panel administrativo, API, app iOS/Android, datos, integraciones, seguridad y operación.

Fuente de verdad: código local de `projects/jahatelo/app/jahatelo-app` y `projects/jahatelo/web/jahatelo-web`. El esquema de Prisma y los Route Handlers prevalecen sobre esta guía si existiera una diferencia.

Seguridad: este documento no contiene credenciales, tokens, datos personales ni configuraciones secretas de producción.

1. Propósito y superficies del producto

Jahatelo permite descubrir moteles, consultar sus habitaciones, tarifas, fotografías, promociones, menú y ubicación, y realizar acciones de contacto. Un mismo backend sirve tres superficies, que deben conservar las mismas reglas de negocio.

Superficie	Proyecto	Público	Responsabilidad
Web pública y PWA	<code>web/jahatelo-web</code>	visitantes y usuarios registrados	búsqueda, mapas, ficha del motel, favoritos, promociones y cuenta
Panel administrativo	<code>web/jahatelo-web/app/admin</code>	superadministración y administradores de motel	operación, catálogo, comercial, finanzas, auditoría y soporte
App móvil	<code>app/jahatelo-app</code>	visitantes y usuarios registrados	experiencia nativa iOS/Android sobre los contratos <code>/api/mobile/*</code>

La regla de evolución principal es: **una regla funcional se define una vez y se verifica en web, admin, API y app antes de considerarla terminada**. No se duplican criterios de plan, precios, visibilidad, moneda, amenities ni ordenamiento en pantallas aisladas.

2. Arquitectura



- Next.js es el backend y la web: no existe un backend móvil independiente.
- PostgreSQL es la fuente transaccional; Prisma es su contrato técnico.
- La app consume una proyección móvil de la API, no consulta la base directamente.
- S3 aloja media subida. Las URLs de medios no deben almacenarse en Git.
- Vercel construye y publica la web. El despliegue de producción debe provenir de `main` mediante el flujo `staging → main`.

3. Estructura de repositorio

```
projects/jahatelo/
├── app/jahatelo-app/           # Expo / React Native
│   ├── screens/               # pantallas y tabs
│   ├── components/           # UI reutilizable
│   ├── services/             # API, cache, prefetch, push y Sentry
│   ├── utils/, hooks/, contexts/ # comportamiento compartido
│   ├── constants/            # diseño, planes y amenities
│   ├── ios/, android/        # proyectos nativos
│   └── docs/SISTEMA_JAHATELO.* # esta documentación
├── web/jahatelo-web/          # Next.js
│   ├── app/                  # páginas, layouts y Route Handlers
│   ├── components/           # UI pública, admin y dominio
│   ├── lib/domain/           # reglas reutilizables de dominio
│   ├── lib/validations/      # contratos Zod
│   ├── prisma/schema.prisma  # modelo PostgreSQL
│   ├── prisma/migrations/    # migraciones versionadas
│   ├── scripts/              # guardas y tareas operativas
│   └── vercel.json            # build y programación declarada
```

4. Stack y convenciones técnicas

Web, administración y API

- Next.js `16.1.6`, React `19.2`, TypeScript y App Router.
- Prisma `6.19` sobre PostgreSQL.
- Tailwind CSS, React Hook Form, Sonner y Lucide.
- Zod para validación de requests; DOMPurify/sanitización para textos.
- `jose` y cookies/tokens para autenticación; `requireAdminAccess` para autorización de API.
- Jest, Playwright, ESLint y TypeScript estricto para calidad.

App nativa

- Expo SDK 54, React Native `0.81`, React `19.1` y React Navigation.
- `expo-image`, `expo-location`, `expo-notifications`, `expo-splash-screen`, Lottie, AsyncStorage, NetInfo, Sentry y `react-native-maps`.
- Bundle/package: `app.jahatelo.mobile`.

- La instalación temporal de iPhone se realiza como **Release nativo con bundle incluido**, no como Development Build con Metro.

Convenciones de código

- Las reglas de precio viven en `lib/domain/motels/pricing.ts` ; las de plan en `lib/domain/motels/planPresentation.ts` .
- El detalle administrativo de motel está dividido por dominio. No volver a concentrar fotos, habitaciones, promociones, menú y ubicación en un componente gigante.
- Toda lista paginada usa `meta.total` y resúmenes del servidor; los contadores no se calculan a partir de la primera página cargada.
- Las acciones destructivas requieren confirmación y auditoría cuando corresponda.
- Las vistas móviles de la web reproducen funcionalmente la app, pero no reutilizan componentes React Native.

5. Dominio y datos

Entidades centrales

Área	Modelos	Función
Catálogo	<code>Motel</code> , <code>RoomType</code> , <code>RoomPhoto</code> , <code>Schedule</code>	ficha, habitaciones, fotos ordenadas y horarios
Tarifas	<code>RoomDayRate</code> , <code>RoomWeekdayRate</code>	precios por grupo de día y excepciones por día/duración
Ubicación	<code>CountryCatalog</code> , <code>CityCatalog</code>	catálogo normalizado de país/ciudad
Amenities y menú	<code>Amenity</code> , <code>RoomAmenity</code> , <code>MenuCategory</code> , <code>MenuItem</code> , <code>Product</code>	amenities por habitación y menú
Usuarios	<code>User</code> , <code>AccessProfile</code> , <code>AccessProfilePermission</code> , <code>Favorite</code> , <code>Review</code>	cuentas, perfiles, permisos, favoritos y reseñas Jahatelo
Comercial	<code>Promo</code> , <code>PromoCode</code> , <code>MotelProspect</code> , <code>HomeBanner</code> , <code>Advertisement</code>	promociones, cupones, prospectos y publicidad
Financiero	<code>PaymentHistory</code> , datos de facturación del motel	cobros, estado y plan
Observabilidad	<code>AuditLog</code> , <code>MotelAnalytics</code> , <code>VisitorEvent</code> , <code>AdAnalytics</code>	auditoría y métricas
Comunicación	<code>PushToken</code> , <code>UserNotificationPreferences</code> , <code>ScheduledNotification</code> , <code>AutoNotificationConfig</code> , <code>ContactMessage</code>	push, preferencias, campañas e inbox

Estados, roles y visibilidad

- Motel: `PENDING` , `APPROVED` , `REJECTED` .
- Un motel se publica solo cuando está `APPROVED` e `isActive=true` .
- Roles base: `SUPERADMIN` , `MOTEL_ADMIN` , `USER` .

- Los perfiles configurables (`AccessProfile`) son la fuente preferida de permisos por módulo y acción (`VIEW` , `CREATE` , `UPDATE` , `DELETE` , `EXPORT` , `MANAGE`). El arreglo legacy `modulePermissions` existe como respaldo para cuentas antiguas.
- `MOTEL_ADMIN` queda restringido a su `motelId` ; no puede cambiar por sí mismo estado/habilitación, país, ciudad, dirección ni enlace de Maps. Tampoco puede eliminarse ni modificarse su propia habilitación.

Catálogo de ubicación

- Países y ciudades se administran desde Configuración por superadministración.
- El motel almacena el texto de país y ciudad ya normalizado contra el catálogo.
- Barrio no forma parte del modelo operativo ni de los formularios actuales.
- La dirección se presenta como una sola línea: `Dirección, Ciudad` .

Fotos y amenities

1. Los amenities son exclusivamente de habitación; el motel publica la unión de los amenities de sus habitaciones activas.
2. La galería pública es la de habitaciones. No existe una galería operativa independiente del motel.
3. `RoomPhoto.order` y `RoomType.order` determinan el orden en admin, web y app.
4. La portada del motel usa `featuredPhotoWeb` para web (16:9), `featuredPhotoApp` para app (4:5) y `featuredPhoto` como compatibilidad.
5. Las cargas se hacen mediante upload manual; los campos de URL de foto no son el flujo normal de edición.

6. Planes y reglas comerciales

Plan	Catálogo público	Contenido publicado	Analytics para administrador de motel
<code>FREE</code>	visible, visualmente atenuado	solo detalles base; sin habitaciones, promos, menú ni reseñas	sin acceso
<code>BASIC</code>	visible	contenido según límites configurados	resumen
<code>GOLD</code>	visible	contenido según límites configurados	completo
<code>DIAMOND</code>	visible con glow comercial	contenido según límites configurados	completo

- El glow se aplica únicamente a `DIAMOND` y debe ser consistente en todas las cards web/app.
- Un motel `FREE` es navegable; no debe quedar como tarjeta bloqueada.
- La regla de plan se aplica tanto en el mapper móvil como en la página pública. Ocultar una tab en UI no sustituye la restricción de datos del backend.

7. Tarifas de habitaciones

Cada habitación admite los bloques `1h` , `1.5h` , `2h` , `3h` , `12h` , `24h` y `Dormida` (no "Noche"). Todos los importes se guardan como enteros PYG y se muestran como `Gs.` sin cálculos que alteren el valor enviado

por admin.

Prioridad de precio efectivo

```
tarifa puntual para día + duración
    ↓ si no existe
tarifa por grupo WEEKDAY / WEEKEND
    ↓ si no existe
tarifa base de habitación
```

- **WEEKDAY** : domingo a jueves; **WEEKEND** : viernes y sábado.
- La hora de referencia es **America/Asuncion** .
- Las excepciones por día se guardan como una fila por **(habitación, día, duración)** ; la restricción única evita dos precios para la misma combinación.
- El mínimo mostrado en cards y detalle se calcula sobre habitaciones activas y tarifas efectivas. Si no existe ningún precio válido se muestra "Consultar"; en caso contrario, "Desde Gs. ...".
- Las vistas de preview de admin muestran tarifas base, por grupo y puntuales ya cargadas antes de editar.

8. Google Maps y ubicación exacta

El campo **mapUrl** acepta tanto el vínculo de **Compartir** de Google Maps como el iframe de **Insertar un mapa**.

1. El formulario normaliza un iframe y persiste solo su **src** ; además extrae coordenadas para mapa/listados.
2. Al abrir Google Maps, una URL normal **/maps/place/...** se preserva completa: contiene la ficha y el pin exacto del negocio.
3. Para un iframe Embed, se intenta obtener el identificador de ficha (CID) y abrir la ficha del negocio.
4. Solo si no existe identidad de ficha se usa latitud/longitud como respaldo.

Regla operativa: preferir siempre el vínculo de "Compartir" para máxima precisión. Nunca convertir un enlace de ficha en un simple **@lat,lng** o **q=lat,lng** , pues esto abre el centro del mapa y puede alejar el pin del negocio.

9. Web pública y PWA

Rutas principales

Ruta	Función
/	inicio, destacados, promos, publicidad y exploración
/search	búsqueda y sugerencias
/mapa , /nearby	mapa y cercanía
/ciudad/[ciudad]	listados por ciudad
/motels/[slug]	detalle del motel
/mis-favoritos , /perfil	cuenta y favoritos
/login , /register	autenticación
/registrar-motel	captación pública
/contacto , /soporte , /privacidad , /terminos	soporte y legales

Detalle de motel

- La ficha reúne detalles, promociones, habitaciones, menú y reseñas según plan y disponibilidad.
- Las fotos de habitación tienen visor ampliado con navegación entre fotos.
- Las tabs se sincronizan con el desplazamiento/gesto móvil sin interferir con la galería.
- Favoritos, contacto y métricas se registran mediante contratos comunes.
- Solo se exponen reseñas de Jahatelo; Google no se usa como fuente de calificación pública.

PWA

La web declara `manifest.json` y registra `sw.js` mediante `PwaRegistrar`. La instalación es opcional y no reemplaza la app nativa. La interfaz de cliente en navegador móvil debe mantener equivalencia funcional con iOS/Android, conservando una presentación de escritorio separada.

10. Panel administrativo

El panel está bajo `/admin`. El acceso se valida en layout, interfaz y Route Handlers; una pantalla oculta nunca es la única barrera de seguridad.

Módulo	Superadministración	Administrador de motel
Dashboard	métricas globales	resumen operativo propio
Gestión de motel	todos los moteles	acceso directo a su motel
Habitaciones, fotos, menú y promos	administra todos	solo su motel
Amenities y ubicación	catálogo global	lectura/uso autorizado según permisos
Prospectos y ciudades	sí	no
Usuarios, perfiles y auditoría	sí	no
Financiero	gestión completa	vista informativa propia
Analytics	todos	propio, sujeto al plan
Canje de códigos	auditoría/global	canje e historial de sus promos
Notificaciones masivas	sí	no puede enviar a clientes

Alta y ciclo comercial de motel

1. Un prospecto se crea manualmente o desde captación, con estado **NEW** , **CONTACTED** , **IN_NEGOTIATION** , **WON** o **LOST** .
2. Desde el prospecto se puede iniciar "Dar de alta motel"; sus datos disponibles se precargan.
3. Al convertir, se completan obligatoriamente los datos requeridos del perfil, ubicación y contacto. El motel nace con el plan elegido (por defecto **FREE**) y puede quedar pendiente/habilitado según el flujo administrativo.
4. Los borradores locales y el antiguo formulario autoguardado fueron retirados: no forman parte del flujo vigente.

Promociones y códigos

- Una promo puede ser global o asociada a un motel, tener vigencia, imagen y reglas de códigos.
- Los códigos tienen estado **PENDING** o **USED** , dispositivo emisor, fecha de canje y responsable.
- El cliente obtiene el código desde la experiencia pública/app; el motel lo valida desde "Canjear código" y consulta su historial.
- El historial debe preservarse al desactivar una promo. Eliminar entidades con relaciones exige revisar explícitamente la política de retención.

Auditoría

AuditLog registra actor, acción, entidad, módulo, método, ruta, estado HTTP, IP, user-agent, metadatos y, cuando aplica, antes/después. El middleware de acceso también deja trazas de acceso concedido/denegado. Toda nueva operación administrativa sensible debe usar el helper de auditoría.

11. App iOS y Android

Experiencia

- Inicio, búsqueda, ciudades, cerca mío, mapa, favoritos, perfil, autenticación y detalle de motel consumen `/api/mobile/*`.
- El detalle usa tabs/scroll sincronizados, pull-to-refresh, galería fullscreen y gestos protegidos para no cambiar de tab al deslizar fotos.
- La app recibe `mapUrl` además de coordenadas; al abrir Maps aplica la misma regla de ficha exacta que la web.
- El splash y launch screen se coordinan visualmente para evitar una transición perceptible.

Datos y resiliencia

- `services/motelsApi.js` normaliza contratos móviles, fotos, amenities, plan, precios y ubicación.
- Cache local, prefetch, reintentos y estado de red se concentran en `cacheService`, `prefetchService`, `useNetworkStatus` y `useOnlineRetry`.
- La app envía `X-App-Version`; el servidor puede responder `426` si hay una versión mínima obligatoria.
- Todos los listados deben implementar actualización manual por arrastre cuando la pantalla lo permita.

Builds y dispositivo

```
cd '/Users/jota/Desktop/AKAHATA_STUDIO/projects/jahatelo/app/jahatelo-app'  
npm run ios:device -- <UDID>
```

Este es el camino preferido para reinstalar rápidamente una Release iOS temporal en un iPhone conectado, sin Metro. Usar una reconstrucción nativa completa solo cuando cambien Pods, permisos, `app.json`, firma, scheme o archivos nativos. Android se distribuye mediante APK generado desde una build nativa actualizada.

12. API y contratos

Prefijo	Responsabilidad
/api/mobile/*	catálogo y acciones de la app: moteles, ciudades, mapa, favoritos, reseñas, auth y preferencias
/api/admin/*	operación autenticada: moteles, habitaciones, fotos, amenities, ubicación, promos, códigos, usuarios, perfiles, prospectos, financiero, analytics, auditoría e inbox
/api/auth/*	login, sesión, registro, Google, correo y teléfono/OTP según flujo
/api/public/*	lectura/captación pública y promociones
/api/upload/*	subida autorizada y flujo S3
/api/analytics/*	eventos de catálogo/visitantes
/api/notifications/*	creación, envío inmediato y consulta de campañas
/api/cron/*	procesadores protegidos de tareas programadas
/api/health	verificación de salud

Reglas de contrato:

1. Validar en el borde con Zod y responder errores estables, sin stack traces ni secretos.
2. Todo cambio de Prisma requiere migración revisada, compatibilidad de API y actualización de mappers web/móvil.
3. La API debe aplicar límites de plan y propiedad de recurso; nunca confiar solo en la UI.
4. `mapUrl`, `latitude` y `longitude` describen una misma ubicación; al cambiar el enlace se recalculan las coordenadas, pero no se reemplaza un enlace de ficha al abrir Google Maps.

13. Autenticación, permisos y seguridad

- JWT firmado y validación de sesión en servidor.
- Roles, perfiles y acciones se verifican mediante `requireAdminAccess`.
- Cuentas de motel se asocian a un motel. No se habilita la autocreación de equipos de motel desde ese perfil.
- Validación Zod, sanitización, límites de carga y rate limiting con Upstash/fallback controlado.
- HTTPS, CORS/CSRF según endpoint y protección de staging se configuran en middleware/entorno.
- Secretos solo en variables de entorno. Todo `NEXT_PUBLIC_*` o `EXPO_PUBLIC_*` es visible en el cliente.
- No almacenar contraseñas, tokens, URLs firmadas ni datos de producción en documentación, commits o capturas.

14. Notificaciones y comunicaciones

- La app registra tokens Expo en `PushToken`; las preferencias viven en `UserNotificationPreferences`.
- Solo superadministración puede programar o enviar notificaciones a usuarios. Un motel no puede enviar campañas a clientes.

- Para notificaciones inmediatas, la API procesa el envío en el momento. Para las programadas, `ScheduledNotification` registra destinatario, categoría, resultado y fallos.
- El código declara tres endpoints de cron: proceso de notificaciones, limpieza de datos y limpieza de códigos. `vercel.json` los programa diariamente a las 03:00, 04:00 y 05:00 UTC respectivamente.
- Si el entorno utiliza un programador externo (por ejemplo Supabase), debe invocar los mismos endpoints protegidos y quedar documentado en la configuración de producción. No afirmar que está activo sin verificarlo en el proveedor.
- SMS usa AWS SNS cuando el flujo lo requiera; correo usa SMTP; Expo Push atiende dispositivos móviles. WhatsApp no es canal operativo automático actual.

15. Integraciones externas

Servicio	Uso	Variables/grupo
PostgreSQL	datos principales	<code>DATABASE_URL</code> , <code>DIRECT_URL</code>
Vercel	build y hosting web	proyecto, rama permitida, variables
AWS S3	fotos y media	bucket, región y credenciales AWS
AWS SNS	SMS/OTP	<code>AWS_SNS_*</code> y credenciales AWS
Google Maps	mapas, iframe/enlaces y geocoding	claves Maps públicas/servidor
Google OAuth	login social	IDs de cliente por plataforma
SMTP	correo	<code>SMTP_*</code> , <code>EMAIL_FROM_*</code>
Expo	push y builds móviles	URL API, proyecto/bundle y tokens
Sentry	fallos y trazas	DSN web/móvil/servidor
Upstash	rate limiting	URL y token REST

Antes de cambiar un proveedor, identificar el que está activo en las variables del entorno objetivo. Cloudinary aparece como configuración heredada/opcional, no debe asumirse como el storage activo si S3 está configurado.

16. Media, rendimiento y observabilidad

1. Validar MIME, tamaño, autorización y resultado de cada upload.
2. Usar variantes correctas de portada por plataforma y evitar estirar/recortar la imagen de origen.
3. Mantener órdenes de foto/habitación estables y actualizar en memoria tras reordenar; no obligar al usuario a recargar.
4. Las listas usan paginación/infinite scroll, deduplicación por ID y contadores del servidor.
5. `MotelAnalytics` , `VisitorEvent` y `AdAnalytics` capturan uso funcional; Sentry captura fallos técnicos.
6. No enviar datos sensibles en eventos, logs ni etiquetas de observabilidad.

17. Desarrollo, pruebas y despliegue

Comandos de validación

```
# web
cd '/Users/jota/Desktop/AKAHATA STUDIO/projects/jahatelo/web/jahatelo-web'
npm run lint
npx tsc --noEmit --incremental false
npm test -- --runInBand
npm run build

# app
cd '/Users/jota/Desktop/AKAHATA STUDIO/projects/jahatelo/app/jahatelo-app'
npm run lint
npm run typecheck
npm test -- --runInBand
```

Base de datos

```
cd '/Users/jota/Desktop/AKAHATA STUDIO/projects/jahatelo/web/jahatelo-web'
npx prisma generate
npx prisma migrate dev --skip-seed # solo desarrollo y previa revisión
npx prisma migrate deploy # entorno controlado de deploy
```

No ejecutar migraciones destructivas, restauraciones ni scripts de limpieza sin backup verificable y aprobación explícita.

Flujo de entrega

1. Revisar `git status`, diff y cambios ajenos antes de editar.
2. Validar el alcance: TypeScript/web, lint/app y tests relevantes.
3. Confirmar cuenta GitHub y Vercel de Jahatelo.
4. Hacer commit en `staging`, push, PR hacia `main`, esperar checks y fusionar desde GitHub.
5. Verificar el despliegue asociado y probar el caso funcional en producción.
6. No hacer commit, push, merge ni deploy sin confirmación explícita. "CPM" significa este flujo completo cuando el responsable lo solicita.

18. Runbook de incidencias

Síntoma	Verificación inicial
Web/API 500	logs Vercel/Sentry, variables, Prisma/DB y endpoint exacto
App sin datos	<code>EXPO_PUBLIC_API_URL</code> , red, contrato <code>/api/mobile/*</code> , caché y 426
App abre Expo launcher	se instaló Development Build; reinstalar Release con bundle nativo
Fotos faltantes o mal encuadradas	referencia S3, permisos, variante 16:9/4:5 y orden
Pin de Maps cercano	preservar enlace de ficha <code>maps/place</code> ; no degradarlo a coordenadas
Push no llega	token activo, permisos, preferencias, registro de resultado y scheduler activo
Admin sin acceso	sesión, rol, perfil, acción requerida, módulo y motel asignado
Precio incorrecto	revisar prioridad puntual → grupo → base y que el monto sea entero PYG
Contador de listado incorrecto	usar <code>meta.summary</code> / <code>meta.total</code> , no la longitud de la página

19. Normas de evolución

1. Diseñar primero el contrato y la regla de dominio; después adaptar web, app y admin.
2. Preferir módulos de dominio, tipos explícitos y componentes pequeños por responsabilidad.
3. No introducir campos de formulario sin una persistencia, permiso, visualización y caso de uso claros.
4. No mantener compatibilidad obsoleta sin fecha de retirada y plan de migración.
5. Toda integración nueva debe documentar credenciales requeridas, permisos, coste, monitoreo, reintentos y rollback.
6. Archivos temporales, recovery y backups locales no son parte del producto: la recuperación se apoya en Git, backups de DB y almacenamiento documentado.

Documento vivo. Actualizar esta guía y regenerar HTML/PDF cuando cambien Prisma, contratos API, permisos, planes, proveedores, build nativo o flujo de despliegue.